

Chapitre 2 – Commencer à programmer

• Note générale sur les fichiers qui accompagnent chaque chapitre

Pour la plupart des chapitres (à l'exception notable du Chapitre 1), nous fournissons un ensemble de Notebooks Jupyter à l'attention des enseignants, en plus des fichiers fournis aux élèves et des corrigés du présent Livre du Professeur (xx est le numéro du chapitre) :

- `cXX-cours-eleve.ipynb` : exemples de code du cours. Ce notebook est principalement destiné aux élèves, afin qu'ils puissent facilement expérimenter avec les constructions illustrées dans le cours.
- `cXX-activite-corrige.ipynb` : corrigé de l'activité du chapitre, destiné aux enseignants.
- `cXX-TP-corrige.ipynb` : corrigé du TP du chapitre, destiné aux enseignants.
- `cXX-exercices-newbie-corrige.ipynb` : corrigés des exercices Newbie, destiné aux enseignants.
- `cXX-exercices-initie-corrige.ipynb` : corrigés des exercices Initié, destiné aux enseignants.
- `cXX-exercices-titan-corrige.ipynb` : corrigés des exercices Titan, destiné aux enseignants.

Certains de ces Notebooks sont assez longs à charger, merci d'être patients ! En principe, les énoncés s'affichent sur fond bleu pour les distinguer des corrections. Si ce n'est pas le cas, il suffit d'exécuter la première cellule (cliquer dans la cellule et taper Majuscule-Entrée).

La plus grande partie des corrigés sont directement exécutables dans Jupyter (en tapant Majuscule-Entrée dans les cellules de code). Pour les exercices qui ne sont pas exécutables dans Jupyter, les fichiers nécessaires (Python ou autres) sont fournis séparément.

Une version Python pure de chacun des ces fichiers est également fournie (même nom mais suffixe `.py`). Ces fichiers Python ne sont pas destinés à être exécutés tels quels, mais à servir de source pour copier-coller le code dans un environnement Python, comme par exemple EduPython.

• Note générale sur les bibliothèques `turtle` et `nsi_turtle`

Plusieurs chapitres, dont le présent Chapitre 2, utilisent la bibliothèque Python `turtle`, qui est en principe intégrée à toute installation Python. Cependant celle-ci ne fonctionne pas bien depuis Jupyter. Aussi fournissons-nous la bibliothèque `nsi_turtle`, destinée à être utilisée à la place de `turtle` à la fois dans Jupyter et directement dans Python. Pour cela le fichier `nsi_turtle.py` doit être dans le même dossier que les fichiers Python ou Jupyter qui l'utilisent.

Dans Jupyter, `nsi_turtle` utilise la bibliothèque `ipyturtle` (<https://github.com/gkvoelkl/ipython-turtle-widget>) qui doit au préalable être installée, soit directement depuis le terminal avec la commande `pip install ipyturtle`, soit depuis EduPython avec la commande « installer un module avec pip » du menu « Outils ». `nsi_turtle` fournit l'ensemble des fonctions de la bibliothèque `turtle` qui sont utilisées dans le manuel. Cependant, pour des raisons techniques, certaines fonctions sont inopérantes : affichage de texte et gestion des événements.

`nsi_turtle` est également utile avec un environnement Python autre que Jupyter, tel que EduPython : Elle améliore la gestion des événements, et configure la tortue pour dessiner instantanément plutôt qu'avec une animation. Elle permet aussi que le même programme peut être utilisé dans Jupyter ou directement en Python.

Notes sur le chapitre

Section 4.1 - Conditionnelles et blocs

Nous n'introduisons pas l'expression conditionnelle `x = y if c else z`, qui nous semble source de confusion à ce stade de l'apprentissage du langage.

Section 4.2 - Boucles

La fonction `random` utilisée dans les exemples doit être importée depuis la bibliothèque `random`:

```
from random import *
```

ou

```
from random import random
```

`random()` retourne un nombre en virgule flottante compris entre 0 (inclus) et 1 (exclus).

Nous n'introduisons pas les instructions `break` et `continue` de Python qui permettent de sortir d'une boucle ou de sauter à la prochaine itération. Il nous semble préférable de maîtriser les boucles sans ces instructions car celles-ci peuvent conduire à de mauvaises habitudes de programmation. Par contre nous utilisons le fait que l'instruction `return` (voir ci-dessous) interrompt l'exécution de la fonction pour retourner immédiatement une valeur. Cela peut encourager une meilleure modularité du code en créant des fonctions simples qui utilisent cette propriété, comme dans l'exemple de la fonction `min` du cours.

Section 5 - Définition de fonctions

Les termes « paramètre » et « argument » sont souvent utilisés de manière interchangeable en informatique. Ici on utilise de préférence le terme « paramètre » pour la définition de la fonction et « argument » pour l'appel. On utilise parfois (mais pas dans ce manuel) les termes de « paramètre formel » pour la définition et « paramètre réel » pour l'appel.

De même le terme « procédure » ou « commande » est parfois utilisé pour des fonctions qui ne retournent pas de valeur (et ont donc seulement des effets de bord). Dans ce manuel nous utilisons uniquement le terme « fonction ». (Techniquement, une fonction Python qui ne retourne pas de valeur retourne la valeur `None`. La valeur `None` est introduite au Chapitre 4.)

Section 5.3 - Variables locales et globales

Les règles de visibilité de Python sont une source d'erreurs même pour les programmeurs chevronnés. Le fait qu'une référence à une variable globale ne nécessite pas de déclarer celle-ci en tant que telle (avec l'instruction `global`) si elle est utilisée seulement en lecture est commode mais dangereux.

Activité : La tortue Python

Tracer un carré

```
forward(100)
right(90)
forward(100)
right(90)
forward(100)
right(90)
forward(100)
```

Utiliser une variable

```
cote = 50
forward(cote)
right(90)
forward(cote)
right(90)
```

```
forward(cote)
right(90)
forward(cote)
```

Utiliser une boucle

1. L'angle est de $2\pi/n$ (ou $360/n$ en degrés).

2.

```
n = 12
cote = 50
angle = 360/n
for i in range(n):
    forward(cote)
    right(angle)
```

Définir et utiliser une nouvelle fonction

1.

```
def polygone(n, cote):
    """ Tracer un polygone a n cotes de longueur "cote" """
    for i in range(n):
        forward(cote)
        right(360/n)
```

2.

```
for i in range(36):
    polygone(3, 100)
    right(10)
```

QCM (CHECKPOINT)

1 b, c et f ; 2 e et f ; 3 a ; 4 a ; 5 a, b, e, h et i ; 6 b et c ; 7 b ; 8 d ; 9 c ; 10 5 ; 11 c ; 12 a, b et c.

TP A : Un carré en ASCII Art

```
def carre(cote):
    """ Affiche un carré """
    # Les lignes de notre dessin
    for i in range(cote):
        if i == 0 or i == cote-1:
            # Première ou dernière ligne : on affiche `cote` fois le symbole 'X '
            print("X " * cote)
        else:
            # Lignes intermédiaires : on affiche seulement le premier et le
            # dernier 'X ', avec des (doubles) espaces entre les deux.
            # Les règles de priorités des opérations + et * s'appliquent.
            print("X " + " " * (cote-2) + "X ")

carre(8)
```

TP B : D'autres formes

```
def carre_croix(cote):
    """ Affiche un carré avec un croix au milieu """
    mediane = cote // 2 # Important à définir quand `cote` n'est pas impair.
```

```

# Les lignes de notre dessin
for i in range(cote):
    if i == 0 or i == cote-1 or i == mediane:
        # Première ou dernière ligne, ou ligne médiane.
        print("X " * cote)
    else:
        # Lignes intermédiaires : on affiche seulement le premier, médian,
        # et dernier 'X', avec des (doubles) espaces entre les deux.
        # Les règles de priorités des opérations + et * s'appliquent.
        print("X " + " "*(mediane-1) + "X " + " "*(cote-mediane-2) + "X ")

carre_croix(9)
print()

def carre_diagonale(cote):
    """ Affiche un carré avec une diagonale """
    # Les lignes de notre dessin.
    for i in range(cote):
        if i == 0 or i == cote-1:
            # Première ou dernière ligne.
            print("X " * cote)
        else:
            # Lignes intermédiaires : on affiche le premier et dernier 'X',
            # ainsi que le 'X' correspondant au numéro de ligne, avec des
            # espaces entre eux.
            print("X " + " " * (i-1) + "X " + " " * (cote-i-2) + "X ")

carre_diagonale(9)
print()

def origami(cote):
    """ Affiche un carré avec une diagonale et une demi-diagonale """
    mediane = cote // 2 # Important à définir quand `cote` n'est pas impair.

    # Les lignes de notre dessin.
    for i in range(cote):
        if i == 0 or i == cote-1:
            # Première ou dernière ligne.
            print("X " * cote)
        else:
            # On construit les lignes restantes caractère par caractère.
            ligne = ""
            for j in range(cote):
                # Extrémités :
                if j == 0 or j == cote-1:
                    ligne += "X "

                # Diagonale montante :
                elif j == cote - i - 1:
                    ligne += "X "

                # Diagonale descendante dans la seconde moitié :
                elif j == i and i >= mediane:
                    ligne += "X "

            else:
                ligne += " "

            print(ligne)

origami(9)

def origami2(cote):
    """ Affiche un carré avec une diagonale et une demi-diagonale """

```

```
# Note : Version alternative, plus "matheuse"
mediane = cote // 2 # Important à définir quand `cote` n'est pas impair.
```

```
# Les lignes de notre dessin.
```

```
for i in range(cote):
    if i == 0 or i == cote-1:
        # Première ou dernière ligne.
        print("X " * cote)
    elif i <= mediane:
        # Première moitié : on n'affiche que la diagonale montante (ainsi
        # que les extrémités de la ligne).
        print("X " + " " * (cote-i-2) + "X " + " " * (i-1) + "X ")
    else:
        # Seconde moitié : on affiche les deux diagonales et les extrémités.
        # Extrémité gauche :
        ligne = "X "
        # Espaces jusqu'à la diagonale montante :
        ligne += " " * (cote-i-2) + "X "
        # Espaces jusqu'à la diagonale descendante :
        ligne += " " * (2 * (i-mediane) - 1) + "X "
        # Espaces jusqu'à l'extrémité droite :
        ligne += " " * (2 * mediane - i - 1) + "X "
        print(ligne)
```

```
origami2(9)
```

```
def triangle(base):
```

```
    """ Affiche un triangle pointant vers le haut """
    mediane = base // 2
```

```
    for i in range(mediane):
        # On construit chaque ligne caractère par caractère.
        ligne = ""
        for j in range(base):
            if j == mediane - i or j == mediane + i:
                ligne += "X "
            else:
                ligne += " "
        print(ligne)
```

```
    # Dernière ligne.
```

```
    print("X " * base)
```

```
triangle(11)
```

```
def triangle_inverse(base):
```

```
    """ Affiche un triangle pointant vers le haut en négatif """
    mediane = base // 2
```

```
    print("X " * (base + 2)) # Première ligne pleine
```

```
    for i in range(mediane+1):
        # On construit chaque ligne caractère par caractère.
        ligne = "X " # On commence toujours par un 'X '.
        for j in range(base):
            # Note : Ici on teste l'appartenance à un intervalle
            if mediane - i <= j <= mediane + i:
                ligne += " "
            else:
                ligne += "X "
        ligne += "X " # On termine toujours par un 'X '
        print(ligne)
```

```

# Dernière ligne.
print("X " * (base + 2)) # Dernière ligne pleine.

triangle_inverse(11)

```

TP C : Le sapin de Noël

```

def sapin(hauteur_feuillage, hauteur_tronc, rayon_tronc):
    """ Affiche la forme générale d'un sapin """
    # Première ligne
    print(" " * hauteur_feuillage + "^")

    for i in range(hauteur_feuillage):
        # On construit chaque ligne caractère par caractère.
        ligne = ""
        for j in range(2 * hauteur_feuillage + 1):
            if j == hauteur_feuillage - i - 1:
                # Côté gauche
                ligne += "/"
            elif j == hauteur_feuillage + i + 1:
                # Côté droit
                ligne += "\\"
            else:
                ligne += " "
        print(ligne)

    # Maintenant le tronc
    nb_espaces_avant_tronc = hauteur_feuillage - rayon_tronc
    ligne = " " * nb_espaces_avant_tronc + "|" * (2 * rayon_tronc + 1)
    for i in range(hauteur_tronc):
        print(ligne)

sapin(9, 4, 2)

def sapin(hauteur_feuillage, hauteur_tronc, rayon_tronc):
    """ Affiche la forme générale d'un sapin avec une étoile au sommet """
    # Première ligne
    print(" " * hauteur_feuillage + "*")

    # Deuxième ligne
    print(" " * hauteur_feuillage + "^")

    for i in range(hauteur_feuillage):
        # On construit chaque ligne caractère par caractère.
        ligne = ""
        for j in range(2 * hauteur_feuillage + 1):
            if j == hauteur_feuillage - i - 1:
                # Côté gauche
                ligne += "/"
            elif j == hauteur_feuillage + i + 1:
                # Côté droit
                ligne += "\\"
            else:
                ligne += " "
        print(ligne)

    # Maintenant le tronc
    nb_espaces_avant_tronc = hauteur_feuillage - rayon_tronc
    ligne = " " * nb_espaces_avant_tronc + "|" * (2 * rayon_tronc + 1)
    for i in range(hauteur_tronc):
        print(ligne)

```

```

sapin(9, 4, 2)

def sapin(hauteur_feuillage, hauteur_tronc, rayon_tronc):
    """ Affiche un sapin avec ses feuilles """
    # Première ligne
    print(" " * hauteur_feuillage + "*")

    # Deuxième ligne
    print(" " * hauteur_feuillage + "^")

    for i in range(hauteur_feuillage):
        # On construit chaque ligne caractère par caractère.
        ligne = ""
        for j in range(2 * hauteur_feuillage + 1):
            if j == hauteur_feuillage - i - 1:
                # Côté gauche
                ligne += "/"
            elif j == hauteur_feuillage + i + 1:
                # Côté droit
                ligne += "\\"
            elif hauteur_feuillage - i <= j <= hauteur_feuillage + i:
                # Feuillage : on alterne ' et '
                if (i + j) % 2 == 1:
                    ligne += " "
                else:
                    ligne += ""
            else:
                ligne += " "
        print(ligne)

    # Maintenant le tronc
    nb_espaces_avant_tronc = hauteur_feuillage - rayon_tronc
    ligne = " " * nb_espaces_avant_tronc + "|" * (2 * rayon_tronc + 1)
    for i in range(hauteur_tronc):
        print(ligne)

sapin(9, 4, 2)

from random import random

def sapin(hauteur_feuillage, hauteur_tronc, rayon_tronc, proba_boule):
    """ Affiche un sapin avec son feuillage et des boules de Noël """

    # Première ligne
    print(" " * hauteur_feuillage + "*")

    # Deuxième ligne
    print(" " * hauteur_feuillage + "^")

    for i in range(hauteur_feuillage):
        # On construit chaque ligne caractère par caractère.
        ligne = ""
        for j in range(2 * hauteur_feuillage + 1):
            if j == hauteur_feuillage - i - 1:
                # Côté gauche
                ligne += "/"
            elif j == hauteur_feuillage + i + 1:
                # Côté droit
                ligne += "\\"
            elif hauteur_feuillage - i <= j <= hauteur_feuillage + i:
                # Feuillage ou boule selon le résultat de `random`
                if random() < proba_boule:

```

```

        ligne += "o"
    else:
        # Feuilleage : on alterne ' et "
        if (i + j) % 2 == 1:
            ligne += "'"
        else:
            ligne += '"'
    else:
        ligne += " "
    print(ligne)

# Maintenant le tronc
nb_espaces_avant_tronc = hauteur_feuilleage - rayon_tronc
ligne = " " * nb_espaces_avant_tronc + "|" * (2 * rayon_tronc + 1)
for i in range(hauteur_tronc):
    print(ligne)

sapin(9, 3, 2, .2)

```

Exercices

Exercices Newbie

1 Chaîne de caractères, nombre à virgule flottante, Booléen, entier, Chaîne de caractères, Chaîne de caractères.

2

```

print(2 + 3*5 + 4)
print((5+2)*3 + 4)
print(4 + 5*(2+3))

```

3

```

a = 10
b = 20
(a+b)/2

```

4 x = 0 : True, False, False, False

x = 10 : False, False, True, True

x = -20 : False, True, True, True

```

# Note : l'exercice est à faire "à la main" !
x = 0
print('x = 0')
print(x < 10 and x > -10)
print(x < -10 or x > 10)
print(x <= 10 and x * x >= 100)
print(x > -25 and x < -5 or x > 5 and x < 25)

print()
x = 10
print('x = 10')
print(x < 10 and x > -10)
print(x < -10 or x > 10)
print(x <= 10 and x * x >= 100)
print(x > -25 and x < -5 or x > 5 and x < 25)

```



```
print()
x = -20
print('x = 0')
print(x < 10 and x > -10)
print(x < -10 or x > 10)
print(x <= 10 and x * x >= 100)
print(x > -25 and x < -5 or x > 5 and x < 25)
```

5 1: Logique.

2: (rien)

3: Logique.

Ou pas.

3: Ou pas.

6 10 fois (s = 45);

5 fois (s = 120);

4 fois (s = 20);

boucle infinie

```
# Note : l'exercice est à faire "à la main" !
n = 0
s = 0
for i in range(10):
    n += 1
    s = s + i
print('n =', n, ', s =', s)

n = 0
s = 1
for i in range(1,6):
    n += 1

    s = s * i
print('n =', n, ', s =', s)

n = 0
s = 0
while s < 20:
    n += 1
    s = s + 5
print('n =', n, ', s =', s)

# Attention : ce programme boucle, il faudra l'interrompre
# soit avec ^C dans Python, soit avec la commande Interrupt dans Jupyter
# (ou le bouton avec un carré noir)
n = 0
s = 1
while s != 100:
    n += 1
    s = s * 2
print('n =', n, ', s =', s)
```

7

```
for i in range(1,11):
    print('7 x', i, '=', i*7)
```

8

```
factorielle = 1
for i in range(1, 11):
```

```
factorielle = factorielle * i
print(factorielle)
```

9 a. `n` vaut 5 à la fin du programme.

b. `n` compte le nombre de fois que l'on peut retirer `b` de `a`. C'est la définition du quotient. (Et `a` vaut le reste de la division à la fin du programme.)

c.

```
n = 0
a = 27
a0 = a # mémoriser la valeur initiale de a
b = 5
while a >= b:
    n = n + 1
    a = a - b
# tester que le résultat est correct
if n != a0 // b:
    print('incorrect !')
```

10

```
def quotient(a, b):
    n = 0
    while a >= b:
        n = n + 1
        a = a - b
    return n

print(quotient(16, 4))
print(quotient(16, 5))
```

11

```
def pair(n):
    return n % 2 == 0

print(pair(10))
print(pair(11))
```

12 a. La fonction `double` ne retourne pas de valeur, ce qui en Python se note `None`. `a` vaut donc `None`.

b. Le programme affiche 42.

13

```
pi = 3.14 # avec un point et pas une virgule
def aire_cercle(rayon): # _ et pas -, : à la fin
    return pi * rayon ** 2 # rayon et pas r, ** et pas *
print(aire_cercle(10)) # indentation, parenthèse fermante
```

Exercices Initié

14 a.

```
d = 149597870700 # mètres (aussi appelé UA = Unité Astronomique)
c = 299792458 # mètres / seconde
t = 40075017 # mètres (à l'équateur)
p = 7874966000 # environ, en 2021
```

b.

```

from math import pi

temps_lumiere_terre_soleil = d / c
vitesse_terre = (2*pi*d) / (365.25*24*60*60)
nb_signal_electrique = c/t
rayon_terre = t / (2*pi)
surface_terre = 4 * pi * rayon_terre*rayon_terre
surface_terre_km2 = surface_terre / 1000000
surface_emergee = surface_terre_km2 * (1-0.707)
densite_moyenne_population = p / surface_emergee

print(temps_lumiere_terre_soleil, 'secondes')
print(vitesse_terre, 'm/s, soit', vitesse_terre/1000*60*60, 'km/h')
print(nb_signal_electrique, 'tours')
print(surface_terre, 'm2, soit', surface_terre_km2/1000000, 'millions de km2')
print(densite_moyenne_population, 'hab/km2')

```

15

```

# Note : il existe d'autres solutions.
# Note 2 : les résultats sont des nombres en virgule flottante
# lorsqu'on utilise la division.
# On peut utiliser la division entière pour avoir des résultats entiers
# (mais alors on ne sait pas si le résultat est réellement entier)
print((3 + 1 - 4) / 2)
print(2*3 - (4 + 1))
print(2 * (3 + 1)/4)
print((4 + 2) / (3 - 1))
print((4 * (3 - 1))/2)
print((3 - 1)/2 + 4)
print(4*2 - 3 + 1)
print(4 + 3 * (2 - 1))
print(4 * (3+1) / 2)
print(4*3 - (2 + 1))
print((4*3 - 2)/1)

```

16

```

bm = " belle Marquise,"
vby = " vos beaux yeux"
vyb = " vos yeux beaux"
mf = " me font"
m = " mourir"
a = " d'amour"
print(bm+vby+mf+m+a)
print(a+m+mf+", "+bm+vby)
print(vyb+a+mf+", "+bm+m)
print(m+vby+", "+bm+a+mf)
print(mf+vyb+m+", "+bm+a)

```

17

```

for i in range(2):
    a = i == 1
    for j in range(2):
        b = j == 1
        print('a =', a, 'b =', b)
        print(' not (a and b) =', not (a and b))
        print(' not a or not b =', not a or not b)
        print(' not (a or b) =', not (a or b))
        print(' not a and not b =', not a and not b)

```

18

```
def xor(a, b):
    """ Retourne vrai si a ou b est vrai mais pas les deux """
    return (a or b) and not (a and b)

print(True, 'xor', True, '=', xor(True, True))
print(True, 'xor', False, '=', xor(True, False))
print(False, 'xor', True, '=', xor(False, True))
print(False, 'xor', False, '=', xor(False, False))
```

19 a.

```
# On définit une fonction pour faciliter le test
def et(a, b):
    return not (not a or not b)

print(True, 'et', True, '=', et(True, True))
print(True, 'et', False, '=', et(True, False))
print(False, 'et', True, '=', et(False, True))
print(False, 'et', False, '=', et(False, False))
```

b.

```
# On définit une fonction pour faciliter le test
def ou(a, b):
    return not (not a and not b)

print(True, 'ou', True, '=', ou(True, True))
print(True, 'ou', False, '=', ou(True, False))
print(False, 'ou', True, '=', ou(False, True))
print(False, 'ou', False, '=', ou(False, False))
```

20

```
t = 25

if t < -10:
    print("Il va faire très froid")
elif t < 0:
    print("Il va faire froid")
elif t < 15:
    print("Il va faire frais")
elif t < 25:
    print("Il va faire bon")
elif t < 30:
    print("Il va faire un peu chaud")
elif t < 35:
    print("Il va faire chaud")
else:
```

21 a.

```
a = 2021 # changer l'année pour tester d'autres valeurs

bissextile = False
if a % 4 == 0 and (a % 100 != 0 or a % 400 == 0):
    bissextile = True

print(bissextile)
```

b.

```
m = 2 # changer le mois pour tester d'autres valeurs
```

```

jours = 31
if m < 7 and m % 2 == 0 or m > 7 and m % 2 == 1:
    jours = 30
if m == 2:
    jours = 28
    if bissextile:
        jours = 29

print(jours)

```

22 a.

```

def est_bissextile(annee):
    """ retourne True si l'annee est bissextile """
    bissextile = False
    if annee % 4 == 0 and (annee % 100 != 0 or annee % 400 == 0):
        bissextile = True
    return bissextile

def nb_jours_mois(mois, annee):
    """ retourne le nombre de jours du mois
    de l'annee donnee """
    jours = 31
    if mois < 7 and mois % 2 == 0 or mois > 7 and mois % 2 == 1:
        jours = 30
    if mois == 2:
        jours = 28
        if est_bissextile(annee):
            jours = 29
    return jours

print('juillet 2022 a', nb_jours_mois(7, 2022), 'jours')
print('février 2020 a', nb_jours_mois(2, 2020), 'jours')

```

b.

```

def nb_jours(jour, mois, annee):
    """ retourne le nombre de jours
    depuis le debut de l'annee """
    njours = 0
    # calculer la somme des nombres de jours des mois précédents
    for m in range(1, mois):
        njours = njours + nb_jours_mois(m, annee)
    return njours + jour

print(nb_jours(1, 2, 2022))
print(nb_jours(14, 7, 2022))
print(nb_jours(31, 12, 2022))
print(nb_jours(31, 12, 2024))

```

c.

```

def jour_semaine(jour, mois, annee, jour0):
    """ retourne le jour de la semaine
    de la date donnee. `jour0` est le
    code du premier jour de l'annee """
    return (jour0 + nb_jours(jour, mois, annee) - 1) % 7

# Le dernier jour d'une année non bissextile est le même que le premier
print(jour_semaine(31, 12, 2021, 4))

```

d.

```
# On calcule d'abord le jour0 de l'année 2022
nouvelan2022 = jour_semaine(31, 12, 2021, 4) + 1

# Maintenant on peut calculer le jour demandé
fetnat2022 = jour_semaine(14, 7, 2022, nouvelan2022)
if fetnat2022 == 3:
    print("Le 14 juillet 2022 est bien un jeudi")
else:
    print("il semble y avoir une erreur...")
```

23

```
for i in range(10):
    print((i+1)*2)

i = 2
while i <= 20:
    print(i)
    i = i + 2
```

24

```
from random import random
print("Ha" + "ha"*round(random()*9))
```

25 a. Note : L'exercice est à faire (au moins dans un premier temps) à la main.

Valeurs après chaque « tour » de boucle, pour $a = 10$, $b = 3$

- $r = 1$, $a = 3$, $b = 1$
- $r = 0$, $a = 1$, $b = 0$

Valeurs après chaque tour de boucle, pour $a = 10$ et $b = 15$

- $r = 10$, $a = 15$, $b = 10$
- $r = 5$, $a = 10$, $b = 5$
- $r = 0$, $a = 5$, $b = 0$

b. Le PGCD est le plus grand nombre qui divise à la fois a et b .

Si a divise b , alors $\text{PGCD}(a, b) = a$. Sinon on peut écrire $a = b \times q + r$ et on montre que $\text{PGCD}(a, b) = \text{PGCD}(b, r)$.

Le programme calcule donc r (reste de la division de a par b) et remplace a par b et b par r jusqu'à ce que le reste soit nul. À la fin de la boucle, a est le PGCD de a et b .

26 a.

```
x = 1
while x < 1000000:
    print(x)
    x = x * 2
```

b.

```
n = 0
x = 1
while x < 1000000:
    x = x * 2
    n = n + 1
print(n)
```

27

```
a = 1234

while a != 0:
    print(a % 10)
    a = a // 10
```

28 a.

```
for i in range(0,16):
    print(i, sin(i*pi/15) * 30)
```

b.

```
x = 0.1
print('=' * round(x * 30))
x = 0.5
print('=' * round(x * 30))
x = 1
print('=' * round(x * 30))
```

c.

```
for i in range(0,16):
    print('=' * round(sin(i*pi/15) * 30))
```

d.

```
for i in range(0,16):
    v = sin(i*3.14*2/15)
    if v > 0:
        print(' '*20 + '='*round(v*20))
    else:
        l = round(-v*20)
        print(' '*(20-l) + '='*l)
```

e.

```
for i in range(0,16):
    v = sin(i*3.14*2/15)
    if v > 0:
        print(' '*20 + ' '* (round(v*20) - 1) + '*')
    else:
        l = round(-v*20)
        print(' '*(20-l) + '*')
```

29

```
# En dehors de Jupyter on peut importer turtle à la place de nsi_turtle
from nsi_turtle import *
```

```
# Importer la fonction `random` du module `random`
from random import random
```

```
dedans = True
while dedans:
    # tourner d'un angle aléatoire et avancer de 10 à 50 pas
    left(360.0*random())
    forward(10.0 + 40.0*random())
    # tester si on est dans la fenêtre
    dedans = abs(xcor()) < 400 and abs(ycor()) < 400
```

30 a.

```

n1 = 0
n2 = 1
print(1, n1)
print(2, n2)
for i in range(98):
    print(i+3, n1 + n2)
    n = n1 # se souvenir de n1 avant de changer sa valeur !
    n1 = n2
    n2 = n + n2

```

b.

```

n1 = 0
n2 = 1
print(1, n1)
print(2, n2)
for i in range(98):
    print(i+3, n1 + n2, (n1 + n2) / n2)
    n = n1 # se souvenir de n1 avant de changer sa valeur !
    n1 = n2
    n2 = n + n2

```

31 a.

```

def Syracuse(n):
    """ Affiche les valeurs successives de la suite de Syracuse """
    while n != 1:
        print(n)
        if n % 2 == 0:
            n = n // 2
        else:
            n = n*3 + 1
        print('1 - fin!')
    Syracuse(123)

```

b.

```

def Syracuse_vol(n):
    """ Retourne la durée de vol de la suite de Syracuse """
    vol = 0
    while n != 1:
        vol = vol + 1
        if n % 2 == 0:
            n = n // 2
        else:
            n = n*3 + 1
    return vol

print(Syracuse_vol(120))
print(Syracuse_vol(123))
print(Syracuse_vol(130))

# BONUS - afficher les durées de vol des 150 premières valeurs de n
# Ceci nécessite la notion de tableau introduite au Chapitre 4
vols = []
for n in range(149):
    vols.append(Syracuse_vol(n+1))
from matplotlib.pyplot import plot, show
plot(vols)
show()

```


c.

```
def Syracuse_altitude(n):
    """ Retourne l'altitude de la suite de Syracuse """
    altitude = 0
    while n != 1:
        if n > altitude:
            altitude = n
        if n % 2 == 0:
            n = n // 2
        else:
            n = n * 3 + 1
    return altitude

print(Syracuse_altitude(120))
print(Syracuse_altitude(123))
print(Syracuse_altitude(130))

# BONUS - afficher les durées de vol des 150 premières valeurs de n
# Ceci nécessite la notion de tableau introduite au Chapitre 4
altitudes = []
for n in range(149):
    altitudes.append(Syracuse_altitude(n+1))
from matplotlib.pyplot import plot, show
plot(altitudes)
show()
```

32 a. Note : Cette question est à réaliser « à la main ».

Ce programme affiche une série de lignes constituées de 0 à 9 caractères *.

```
*
**
***
****
*****
*****
*****
*****
*****
*****
```

b.

```
for i in range(10):
    print('*' * i)
```

c.

```
for i in range(10):
    print('*' * (9 - i))
```

33 a.

```
total = 5
for i in range(1, 7): # valeur du premier dé
    for j in range(1, 7): # valeur du second dé
        if i+j == total: # vérifier si on a le total voulu
            print(i, j)
```

Note : Ce programme n'est pas efficace car il teste beaucoup de possibilités inutiles.

Une fois connue la valeur du premier dé (i), on peut calculer la valeur du second. On évite ainsi la deuxième boucle imbriquée :

```
total = 5
for i in range(1, 7): # valeur du premier dé
    j = total - i      # valeur du second dé
    if j > 0 and j < 7: # s'assurer que c'est une valeur possible
        print(i, j)
```

Note : On pourrait encore améliorer l'algorithme lorsque total est inférieur à 6 : la première boucle for peut alors s'arrêter à total.

b.

```
for total in range(2, 13): # totaux possibles entre 2 et 12
    print('total =', total)
    for i in range(1, 7):
        j = total - i      # valeur du second dé
        if j > 0 and j < 7: # s'assurer que c'est une valeur possible
            print(i, j)
```

c.

```
for total in range(3, 19):
    print('total =', total)
    for i in range(1, 7):
        for j in range(1, 7):
            k = total - i - j
            if k > 0 and k < 7:
                print(i, j, k)
```

d.

```
print('Avec 2 dés')
for total in range(2, 13): # totaux possibles entre 2 et 12
    n = 0
    for i in range(1, 7):
        j = total - i      # valeur du second dé
        if j > 0 and j < 7: # s'assurer que c'est une valeur possible
            n = n + 1
    print('total = ', total, ',', n, 'possibilités')

print()
print('Avec 3 dés')
for total in range(3, 19):
    n = 0
    for i in range(1, 7):
        for j in range(1, 7):
            k = total - i - j
            if k > 0 and k < 7:
                n = n + 1
    print('total = ', total, ',', n, 'possibilités')
```

34 a.

```
prix = 127
if prix // 50 > 0:
    print(prix // 50, 'billets de 50€')
    prix = prix % 50

if prix // 20 > 0:
    print(prix // 20, 'billets de 20€')
    prix = prix % 20

if prix // 10 > 0:
    print(prix // 10, 'billets de 10€')
    prix = prix % 10
```

```

if prix // 5 > 0:
    print(prix // 5, 'billets de 5€')
    prix = prix % 5

if prix > 0:
    print(prix, 'pièces de 1€')

```

b.

```

a_rendre = paiement - prix

# nombre de billets de 50
n50 = a_rendre // 50
if n50 > nb_50euros:
    # ne pas utiliser plus de billets que disponible
    n50 = nb_50euros
if n50 > 0:
    print(n50, 'billets de 50€')
    a_rendre = a_rendre - n50*50
    nb_50euros = nb_50euros - n50

# nombre de billets de 20
n20 = a_rendre // 20
if n20 > nb_20euros:
    # ne pas utiliser plus de billets que disponible
    n20 = nb_20euros
if n20 > 0:
    print(n20, 'billets de 20€')
    a_rendre = a_rendre - n20*20
    nb_20euros = nb_20euros - n20

# nombre de billets de 10
n10 = a_rendre // 10
if n10 > nb_10euros:
    # ne pas utiliser plus de billets que disponible
    n10 = nb_10euros
if n10 > 0:
    print(n10, 'billets de 10€')
    a_rendre = a_rendre - n10*10
    nb_10euros = nb_10euros - n10

# nombre de billets de 5
n5 = a_rendre // 5
if n5 > nb_5euros:
    # ne pas utiliser plus de billets que disponible
    n5 = nb_5euros
if n5 > 0:
    print(n5, 'billets de 5€')
    a_rendre = a_rendre - n5*5
    nb_5euros = nb_5euros - n5

# nombres de pièces de 1€
n1 = a_rendre
if n1 > nb_1euro:
    # ne pas utiliser plus de pièces que disponible
    n1 = nb_1euro
if n1 > 0:
    print(n1, 'pièces de 1€')
    a_rendre = a_rendre - n1
    nb_1euro = nb_1euro - n1

if a_rendre > 0: # on n'a pas pu rendre la monnaie

```

```
print('Pas suffisamment de monnaie !')
print('Il manque', a_rendre, '€')
```

c. S'il n'y a pas assez de monnaie, on s'en rend compte seulement à la fin. Il faut donc remettre l'argent dans la caisse. Les variables `n50`, `n20`, etc. permettent de mettre à jour la caisses :

```
if a_rendre > 0: # on n'a pas pu rendre la monnaie
    print('Pas suffisamment de monnaie !')
    print('Il manque', a_rendre, '€')
    # on remet la monnaie rendue dans la caisse
    nb_50euros = nb_50euros + n50
    nb_20euros = nb_20euros + n20
    nb_10euros = nb_10euros + n10
    nb_5euros = nb_5euros + n5
    nb_1euro = nb_1euro + a_rendre
```

d.

```
a_rendre = paiement - prix
```

```
def rendre(a_rendre, valeur, nombre):
    """ Retourne le nombre de pieces ou billets de la valeur à rendre en fonction
        du nombre de pièces ou de billets de cette valeur dans la caisse
    """
    n = a_rendre // valeur
    if n > nombre:
        # ne pas utiliser plus de billets que disponible
        n = nombre
    if n > 0:
        print(n, 'fois', valeur, '€')
        # NOTE : on ne PEUT PAS mettre à jour `a_rendre` et `nombre` ici
        # pour mettre à jour la caisse, car il s'agit d'un paramètre,
        # donc changer la valeur de `nombre` ne change PAS
        # la variable passée en paramètre (par exemple nb_50euros),
        # et de même pour la valeur de `a_rendre` (même si elle porte
        # le même nom que la variable globale `a_rendre`).
    return n

n50 = rendre(a_rendre, 50, nb_50euros)
a_rendre = a_rendre - n50 * 50
nb_50euros = nb_50euros - n50

n20 = rendre(a_rendre, 20, nb_20euros)
a_rendre = a_rendre - n20 * 20
nb_20euros = nb_20euros - n20

n10 = rendre(a_rendre, 10, nb_10euros)
a_rendre = a_rendre - n10 * 10
nb_10euros = nb_10euros - n10

n5 = rendre(a_rendre, 5, nb_5euros)
a_rendre = a_rendre - n5 * 5
nb_5euros = nb_5euros - n5

n1 = rendre(a_rendre, 1, nb_1euro)
a_rendre = a_rendre - n1
nb_1euro = nb_1euro - n1

if a_rendre > 0: # on n'a pas pu rendre la monnaie
    print('Pas suffisamment de monnaie !')
    print('Il manque', a_rendre, '€')
```

35 a.

```
def interets_annuels(montant, taux):
    return montant * taux / 100
```

b.

```
def epargne(montant, taux, duree):
    for i in range(duree):
        montant = montant + interets_annuels(montant, taux)
    return montant
```

c.

```
print(epargne(10000, 1.5, 10))
```

36 L'instruction `print(a)` dans le premier programme est incorrecte car `a` n'est pas définie.

Le second programme est correct et affiche 10, 100, 200.

37 a. Erreurs de syntaxe : Il manque `:` à la fin de la première ligne et à la fin de la ligne `for i`.

Erreur de logique : L'expression `1/i*i` vaut toujours 1 car `\` et `*` ont la même priorité et dont elle est interprétée de gauche à droite : `(1*i)/i`.

Erreur de logique : La dernière ligne ne retourne pas de valeur.

Erreur d'exécution : on calcule `1/i` pour `i` valant 0, et provoque une division par zéro.

b.

```
def somme_inv_carres(n):
    s = 0
    for i in range(1, n+1):
        s = s + 1/(i*i)
    return s

# On définit une fonction pour comparer le résultat
# de la somme des inverses des carrés à  $\pi^2/6$ 
from math import pi
def comparer(n):
    s = somme_inv_carres(n)
    v = pi*pi/6
    print(s, v, s - v)

comparer(100)
comparer(10000)
comparer(1000000)
```

38 a.

```
compteur = 0

def f(n):
    global compteur # ne pas oublier !!
    compteur = compteur + 1
    print(n)

for i in range(100):
    if random() < 0.5:
        f(i)

print('compteur =', compteur)
```

b.

```
compteur = 0
total = 0

def f(n):
    global compteur, total # ne pas oublier !!
    compteur = compteur + 1
    total = total + n
    print(n)

for i in range(100):
    if random() < 0.5:
        f(i)

print('compteur =', compteur)
if compteur > 0:
    print('moyenne =', total/compteur)
```

c.

```
def raz():
    global compteur, total
    compteur = 0
    total = 0

raz()
```

Exercices Titan

39 a. Contrairement à ce à quoi on pouvait s'attendre, la boucle `while` ne s'arrête pas après la 11-ième itération : elle semble infinie. En observant la trace des valeurs produites, on peut observer l'inexactitude des nombres flottants en Python (et des opérations que l'on fait avec). En particulier, $x < 1.0$ au début de la 11-ième itération, et $x > 1.0$ au début de la 12ème itération :

```
...
0.9999999999999999 (11-ième itération)
1.0999999999999999 (12-ième itération)
...
```

Étant donné que x n'est pas exactement égal à 1.0 au début de 11-ième itération, celle-ci s'exécute, et l'instruction $x = x + 0.1$ fait prendre à x une valeur strictement supérieure à 1.0. À partir de la 12-ième itération, la valeur de x étant strictement supérieure à 1.0, puisque celle-ci ne peut que croître, on peut en déduire que la condition d'arrêt de la boucle `while` ne sera jamais vraie lors des itérations suivantes : **la boucle est donc infinie.**

b.

```
x = 0.0
while x <= 1.0:
    print(x)
    x = x + 0.1

# Avec cette version-ci, la boucle s'arrête bien après 11 itérations.
```

c.

```
x = 0.0
while x <= 3.0:
```

```
print(x)
x = x + 0.1
```

La dernière valeur de x est désormais 2.9 (environ).

*# En supposant que l'on souhaite que ce programme compte jusqu'à 3 (inclus)
par incréments de 0.1,
cela signifie qu'il manque la dernière itération (celle affichant 3.0).*

d. Afin de contourner ce problème causé par la représentation des nombres à virgule flottante dans un ordinateur, on peut faire le choix de définir une nouvelle façon de tester si deux nombres à virgule flottants sont égaux, de façon à ce que deux nombres suffisamment proches l'un de l'autre soient considérés comme égaux.

*# Le seuil de différence entre deux nombres en dessous duquel
les deux nombres sont considérés comme égaux est souvent noté epsilon :*
EPSILON = 1e-9

```
def sont_egaux(n1, n2):
    """Retourne True si n1 et n2 sont suffisamment proches pour être
    considérés comme égaux, et False dans le cas contraire.
    """
    return abs(n2 - n1) < EPSILON
```

```
def est_inferieur_ou_egal(n1, n2):
    """Retourne True si n1 est inférieur à n2 ou suffisamment proche
    pour être considéré égal à n2, et False dans le cas contraire.
    """
    return n1 < n2 or sont_egaux(n1, n2)
```

Avec cette nouvelle façon de tester si deux nombres sont égaux et cette valeur de la constante epsilon, les nombres 3.0 et 3.00000000000000013 (la valeur de x dans l'itération manquante de la question précédente) sont désormais considérés égaux, et le programme fonctionne dans les deux cas.

Premier cas (de 0 à 1.0 inclus).

```
x = 0.0
while est_inferieur_ou_egal(x, 1.0):
    print(x)
    x = x + 0.1
```

Deuxième cas (de 0 à 3.0 inclus).

```
x = 0.0
while est_inferieur_ou_egal(x, 3.0):
    print(x)
    x = x + 0.1
```

40 a.

```
def est_premier(n):
    """Retourne True si n est premier, et False dans le cas contraire."""

    # Si n est égal à 0 ou à 1, on conclut immédiatement que n n'est pas premier.
    if n == 0 or n == 1:
        return False

    # Sinon, on teste tous les diviseurs potentiels de n dans l'intervalle [2, n-1].
    n_est_premier = True

    for diviseur_potentiel in range(2, n):
        if n % diviseur_potentiel == 0:
            n_est_premier = False
```

```

    return n_est_premier

# Tests devant retourner True (les nombres sont premiers) :
print(est_premier(2))
print(est_premier(3))
print(est_premier(97))

# Tests devant retourner False (les nombres ne sont pas premiers) :
print(est_premier(1))
print(est_premier(4))
print(est_premier(537))

```

b.

```

def est_premier(n):
    """Retourne True si n est premier, et False dans le cas contraire."""

    # Si n est égal à 0 ou à 1, on conclut immédiatement que n n'est pas premier.
    if n == 0 or n == 1:
        return False

    # Sinon, on teste tous les diviseurs potentiels de n dans l'intervalle [2, n-1].
    diviseur_potentiel = 2

    while diviseur_potentiel < n:
        if n % diviseur_potentiel == 0:
            return False
        diviseur_potentiel = diviseur_potentiel + 1

    return True

```

c.

```

def est_premier(n):
    """Retourne True si n est premier, et False dans le cas contraire."""

    # Si n est égal à 0 ou à 1, on conclut immédiatement que n n'est pas premier.
    if n == 0 or n == 1:
        return False

    # Sinon, on teste tous les diviseurs potentiels de n
    # dans l'intervalle [2, sqrt(n)].
    diviseur_potentiel = 2
    plus_grand_diviseur_potentiel = sqrt(n)

    while diviseur_potentiel <= plus_grand_diviseur_potentiel:
        if n % diviseur_potentiel == 0:
            return False
        diviseur_potentiel = diviseur_potentiel + 1

    return True

```

d.

```

def affiche_nombres_premiers(m):
    """Affiche les m premiers nombres premiers."""

    n = 1
    nb_premiers_affiches = 0

    # Tant que l'on n'a pas affiché m nombres premiers,
    # on teste les nombres un à un et affiche chaque nombre qui est premier.
    while nb_premiers_affiches < m:

```



```

# Si n est premier, on l'affiche, et on incrémente
# le nombre de nombres premiers déjà affichés.
if est_premier(n):
    print(n)
    nb_premiers_affiches = nb_premiers_affiches + 1

# On passe au nombre suivant.
n = n + 1

# Test affichant la liste des nombres premiers compris entre 1 et 100 :
affiche_nombres_premiers(25)

```

41 a.

```

# On suppose que les signes sont représentés par les chaînes de caractères
# "pierre", "papier", et "ciseaux".

def compare_deux_signes(signe_joueur_a, signe_joueur_b):
    """Retourne 1 si le signe du joueur A a l'avantage sur celui du joueur B,
    -1 si c'est l'inverse, et 0 s'il y a égalité.
    """

    # Si les deux signes sont identiques, on retourne 0 (égalité).
    if signe_joueur_a == signe_joueur_b:
        return 0

    # Sinon, on teste si le joueur A gagne, et on retourne 1 si c'est le cas.
    # Note : attention aux parenthèses, qui sont importantes pour écrire le test
    # sur plusieurs lignes dans la condition du if !
    if ((signe_joueur_a == "pierre" and signe_joueur_b == "ciseaux")
        or (signe_joueur_a == "papier" and signe_joueur_b == "pierre")
        or (signe_joueur_a == "ciseaux" and signe_joueur_b == "papier")):
        return 1

    # Si on atteint cette étape, cela signifie que le joueur A n'a pas gagné
    # ET qu'il n'y a pas égalité :
    # on peut donc en conclure que le joueur B gagne, et on retourne -1.
    return -1

# Quelques tests :

# Tests devant retourner 1 (victoire du joueur A)
print(compare_deux_signes("pierre", "ciseaux"))
print(compare_deux_signes("papier", "pierre"))
print(compare_deux_signes("ciseaux", "papier"))

# Tests devant retourner 0 (égalité)
print(compare_deux_signes("pierre", "pierre"))
print(compare_deux_signes("papier", "papier"))
print(compare_deux_signes("ciseaux", "ciseaux"))

# Tests devant retourner -1 (victoire du joueur B)
print(compare_deux_signes("ciseaux", "pierre"))
print(compare_deux_signes("pierre", "papier"))
print(compare_deux_signes("papier", "ciseaux"))

```

b.

```

from random import random

def tire_signe_au_hasard():
    """Retourne un des trois signes possibles du jeu
    en le choisissant au hasard.
    """

```

```

nombre_aleatoire = random() # nombre compris entre 0 et 1

if nombre_aleatoire < 1/3:
    return "pierre"
elif nombre_aleatoire < 2/3:
    return "papier"
else:
    return "ciseaux"

def simule_une_partie_avec_gagnant():
    """
    Simule une partie de jeu et retourne un nombre entier dont le signe indique
    qui a gagné (positif pour le joueur A, négatif pour le joueur B)
    et dont la valeur absolue indique le nombre de points obtenus.
    En cas d'égalité, le nombre de points en jeu augmente de 1, et la simulation
    continue jusqu'à ce que l'un des joueurs perde.
    """

    nb_points_en_jeu = 1

    # Tant que les deux signes tirés au hasard produisent une égalité,
    # on recommence l'opération en incrémentant le nombre de points en jeu de 1.
    comparaison_des_signes = compare_deux_signes(tire_signe_au_hasard(), tire_signe_au_hasard())

    while comparaison_des_signes == 0:
        nb_points_en_jeu = nb_points_en_jeu + 1
        comparaison_des_signes = compare_deux_signes(tire_signe_au_hasard(), tire_signe_au_hasard())

    # Si on atteint cette étape, cela signifie que comparaison_des_signes vaut
    # soit 1 (victoire du joueur A), soit -1 (victoire du joueur B).
    # La valeur à retourner correspond donc au produit de cette valeur avec
    # le nombre de points en jeu.
    return comparaison_des_signes * nb_points_en_jeu

```

c.

```

score_joueur_a = 0
score_joueur_b = 0

nb_parties_a_simuler = 50

# On simule nb_parties_a_simuler (en débutant par la partie numéro 1, et non 0).
for numero_partie in range(1, nb_parties_a_simuler + 1):
    resultat_partie = simule_une_partie_avec_gagnant()

    # Le nombre de points gagnés par l'un des joueurs
    # est la valeur absolue du résultat de la partie.
    score = abs(resultat_partie)

    # On attribue ces points au joueur A ou au joueur B
    # en fonction du signe du résultat de la partie.
    if resultat_partie > 0:
        score_joueur_a = score_joueur_a + score
    else:
        score_joueur_b = score_joueur_b + score

    # On affiche des informations sur la partie que l'on vient de simuler,
    # dont le nombre d'égalités ayant menées à répéter la partie (le cas échéant).
    if score > 1:
        print("Partie", numero_partie, ":", "A", score_joueur_a, "-",
              score_joueur_b, "B", score - 1, "égalité(s)")
    else:

```

```
print("Partie", numero_partie, ": A", score_joueur_a, "-",
      score_joueur_b, "B")
```

42 a.

```
# On importe la fonction sinus et la constante PI (utiles pour cet exercice).
from math import sin, pi

# On définit la fonction f et la valeur V ici :

def f(x):
    """Fonction dont on souhaite estimer l'antécédent x
    étant donné une image f(x)."""
    return sin(x)

V = 0.5

min = 0
max = 1
pas = 0.1

x = min
estimation_antecedent_trouvee = False

while x <= max and estimation_antecedent_trouvee == False:
    # On vérifie si V est dans l'intervalle ]f(x), f(x + pas)[.
    # Si c'est le cas, on assigne True à la variable estimation_antecedent_trouvee
    # afin de stopper la boucle while.
    # Sinon, on incrémente x du pas indiqué (et la boucle continue).
    if f(x) < V and f(x + pas) > V:
        estimation_antecedent_trouvee = True
    else:
        x = x + pas

# Si un antécédent x de V par la fonction f a été trouvé
# dans l'intervalle [min, max], on l'affiche.
# Sinon, on affiche un message indiquant qu'il n'a pas été trouvé.
if estimation_antecedent_trouvee:
    print(x)
else:
    print("Aucun antécédent de V n'a été trouvé dans l'intervalle [", min, max, "]")

# En exécutant le code pour f = sin et V = 0.5, on trouve la valeur 0.5.
```

b.

```
# On commence ici avec x = 0.5, tel qu'on l'a trouvé dans la question précédente.
x = 0.5

min = x
max = x + 0.1
pas = 0.01

estimation_antecedent_trouvee = False

# Le code ci-dessous est identique à celui de la question précédente.
while x <= max and estimation_antecedent_trouvee == False:
    if f(x) < V and f(x + pas) > V:
        estimation_antecedent_trouvee = True
    else:
        x = x + pas

if estimation_antecedent_trouvee:
    print(x)
```

```

else:
    print("Aucun antécédent de V n'a été trouvé dans l'intervale [", min, max, "]")

```

En exécutant le code pour $f = \sin$ et $V = 0.5$, on trouve la valeur 0.52.

c.

```

# Les valeurs des variables min, max et pas sont amenées à être modifiées
# dans la boucle externe du code ci-dessous.
min = 0
max = 1
pas = 0.1

x = min
estimation_antecedent_trouvee = False

# Afin de décider si la précision de l'estimation est suffisante, on peut tester
# si la différence entre l'avant-dernière estimation et la dernière estimation
# est inférieure à un certain seuil.
min_diffERENCE_entre_estimations = 1e-6

def precision_estimation_est_suffisante(derniere_estimation, avant_derniere_estimation):
    return abs(derniere_estimation - avant_derniere_estimation) < min_diffERENCE_entre_estimations

# Afin que la première estimation se déroule correctement, il faut donc choisir
# la valeur initiale de estimation_precedente telle que
# abs(x - estimation_precedente) >= min_diffERENCE_entre_estimations
# lors de la première itération de la boucle extérieure.
estimation_precedente = x + (2 * min_diffERENCE_entre_estimations)

# Boucle externe : on recommence l'estimation jusqu'à obtenir une précision suffisante.
while estimation_antecedent_trouvee == False:
    # Boucle interne : on parcourt l'intervalle [min, max] pour estimer x.
    while x <= max and estimation_antecedent_trouvee == False:
        if f(x) < V and f(x + pas) > V:
            estimation_antecedent_trouvee = True
        else:
            x = x + pas

    # On vérifie si la précision de l'estimation est suffisante :
    # si c'est le cas, on arrête la boucle externe (il n'y a rien à faire de plus) ;
    # sinon on adapte min et max, réduit le pas, et on recommence la boucle interne
    if not precision_estimation_est_suffisante(x, estimation_precedente):
        estimation_antecedent_trouvee = False
        min = x
        max = x + pas
        pas = pas / 10 # on choisit un pas dix fois plus petit

    estimation_precedente = x
    x = min

if estimation_antecedent_trouvee:
    print(x)
else:
    print("Aucun antécédent de V n'a été trouvé dans l'intervale [", min, max, "]")

# En exécutant le code pour  $f = \sin$  et  $V = 0.5$ , on trouve 0.5235979999999998.

```

d.

*# Le code ci-dessus nous a permis de trouver la valeur $x = 0.5235979999999998$.
 # On vérifie donc que 0.5235979999999998 est proche de $\pi/6$:*

```

estimation = 0.5235979999999998

print("Estimation de x dans sin(x) = 0.5 :", estimation)
print("Valeur de sin(PI/6) :", sin(pi/6))

difference = abs((pi/6) - estimation)
print("Différence entre l'estimation et PI/6 :", difference)

# La différence entre l'estimation et l'antécédent réel est très faible :
# 7.755982990298449e-07.
# On peut également noter qu'elle est bien inférieure à la constante 1e-6
# choisie plus tôt. On conclut donc que l'estimation a bien fonctionné.

```

43 Oui, les deux programmes sont équivalents : les deux contiennent une boucle de 10 itérations opérant sur une variable `i`, affichée à chaque tour, débutant à 0, et augmentant de 1 à chaque itération.

```

for i in range(10):
    i = i * 2
    print(i)

# Affiche 0, 2, et les 8 multiples de deux suivants (4, 6, 8...).

i = 0
while i < 10:
    i = i * 2
    print(i)
    i = i + 1

# Affiche 0, 2, 6, 14.

```

Lorsqu'on insère `i = i * 2` avant `print(i)`, les deux programmes ne sont plus équivalents : on peut l'observer en évaluant les deux morceaux de code ci-dessus, qui affichent deux suites de nombres différentes.

Dans le premier cas (boucle `for`), le nombre d'itérations ne varie pas, et les valeurs affichées sont doublées.

Cela vient du fait que l'expression `range(10)` n'est pas affectée par le fait que l'on modifie la variable `i` dans le corps de la boucle. Au début de la n -ième itération, la variable `i` prend toujours la valeur n , peu importe la façon dont cette variable a été modifiée lors de l'itération précédente. Pour cette raison, la boucle produit autant de valeurs que précédemment, et celles-ci sont simplement doublées par l'instruction `i = i * 2` avant d'être affichées.

Dans le second cas (boucle `while`), le nombre d'itérations passe de 10 à 4, et la relation entre les valeurs affichées est moins évidente.

Cela vient du fait que la condition d'arrêt de la boucle (`i < 10`) dépend de la valeur de `i`, qui n'est pas "réinitialisée" à n au début de la n -ième itération.

À chaque itération, la valeur de `i` est doublée, affichée, puis incrémentée de 1. C'est cela qui définit la relation moins évidente entre les valeurs affichées :

```

(itération 1) 0 * 2 = 0
(itération 2) (0 + 1) * 2 = 1*2 = 2
(itération 3) (2 + 1) * 2 = 3*2 = 6
(itération 4) (6 + 1) * 2 = 7*2 = 14

```

La valeur de i augmente ainsi plus rapidement et dépasse 10 lors de la quatrième itération : la condition d'arrêt devient donc vraie, et la boucle se termine avant la cinquième itération, après n'avoir affiché que 4 valeurs.